Detection and Recovery of UE Context Desynchronization in 5G RAN–Core Networks: A Simulation-Based Approach

Sandesh
Dept. of Information Technology
National Institute of Technology Karnataka
Surathkal, India
231IT061

Sujan H
Dept. of Information Technology
National Institute of Technology Karnataka
Surathkal, India
231IT076

Laxminarayan Sahu
Dept. of Information Technology
National Institute of Technology Karnataka
Surathkal, India
231IT035

***Abstract*—Fifth-generation mobile networks distribute session and resource context between the Radio Access Network (RAN) and Core. Partially completed procedures can leave these views inconsistent even when transport is reliable. This paper presents a protocol-aware verifier for a scoped, abstract RAN–Core model. The verifier compares timestamped node snapshots using semantic predicates, freshness and temporal-alignment checks, pending-procedure guards, and evidence persistence. A SimPy prototype models session establishment, QoS modification and release, and evaluates a bounded reconciliation adapter. Across 1,920 benchmark runs, the aware detector identifies all 167 QoS completion-failure episodes with zero false alarms. Under benign delay it produces no false alarms in 240 negative sessions, compared with 225 sessions for a fixed 25 ms checker. For QoS completion failures, mean detection delay is 20.93 virtual ms and mean lead time before a qualifying failed consistency probe is 18.35 ms. Recovery reduces failed probes from 334 to 148 out of 720 and total mismatch exposure from 15,698 to 6,192 session-ms. A further 720 held-out runs characterize timing sensitivity. Paired seed-level bootstrap intervals quantify variability in the recovery benefit. These measurements establish behavior within the declared simulation; they do not measure actual call drops, full handover, or operator-network accuracy.**

***Index Terms*—5G, UE context, RAN–Core consistency, protocol-aware verification, context recovery, discrete-event simulation**

I. Introduction

The 5G architecture distributes responsibilities among the gNB, Access and Mobility Management Function (AMF), Session Management Function (SMF), and User Plane Function (UPF). Session establishment, modification and release involve coordinated resource changes [2], [3]. A node can apply a requested configuration before another node completes the corresponding operation. If completion processing fails, their committed views may disagree.

This problem must be distinguished from ordinary packet loss. NGAP signaling uses reliable SCTP transport [4]. Our fault model therefore targets application request or completion handling and delayed telemetry. It does not assume that an isolated lost SCTP packet permanently removes a signaling message. N2 connects RAN and AMF control functions; N3 is the gNB–UPF user plane; N4 carries SMF–UPF control. The prototype combines Core responsibilities and does not implement an independent N4 or UPF state machine.

Aggregate monitoring can identify abnormal operational behavior, but a per-session consistency checker asks a more specific question: do the observed applied and authorized resource states satisfy a declared relationship? During valid procedures, temporary differences are expected. A detector that merely compares snapshots, or waits a fixed short time, can mistake these transitions for faults.

This project contributes an executable context model, an explainable verifier and a reproducible evaluation of bounded reconciliation. We make no claim of worldwide novelty or deployment readiness.

II. Literature Review

A. Proposal Base Paper

The proposal selected Cho et al.'s framework for AI-based large-scale time-series anomaly detection in 5G Core networks [1]. Its monitoring task concerns KPI streams. Point-level anomaly amplification improves sensitivity to early deviations, while distribution-aware thresholds adapt the decision rule to individual streams. This motivates timely monitoring but differs from our per-session semantic task.

Table I compares scope, not reproduced performance. We do not train the cited model or claim superiority on general anomalies.

B. Consistency and Protocol References

Sid-Otmane et al. discuss data consistency in the 5G specification [5]. This provides additional context for treating distributed UE state as a correctness problem. Our implementation is not a replication of that paper. 3GPP TS 23.502 and TS 38.413 provide procedure background [2], [3]; TS 38.300 provides access and mobility context [6]. The prototype abstracts these references rather than implementing all normative transitions.

TABLE I
Comparison with the Proposal Base Paper

Aspect	Base paper [1]	This prototype
Input	Core KPI time series	Node-state snapshots
Decision	Statistical anomaly	Semantic contradiction
Granularity	KPI stream	UE/session pair
Evaluation	Operator-network study	Synthetic controlled faults
Recovery	Not reproduced here	Custom reconciliation

III. System Model and Methodology

A. Scope and State Representation

For each key $k = (\text{UE}, \text{session})$, the simulator stores independent RAN and Core records

$$C_x = (s_x, a_x, q_x, r_x, c_x, p_x, \tau_x, d_x, v_x, t_x), \tag{1}$$

where s denotes logical session existence, a access resources, q an abstract QoS profile identifier, r access state, c local cell, p pending operation, τ transaction identity, d deadline, v version, and t last-change time. QoS integers are not QFI or 5QI encodings.

The model is a timed extended finite-state machine (EFSM). The RAN resource control has ABSENT and APPLIED(q) states. Core procedure control has STABLE, SETUP_PENDING, MODIFY_PENDING and RELEASE_PENDING states. The committed Core record remains unchanged during a pending operation. Transaction identifiers and timestamps augment the finite control.

Both records initially have no session or resources and QoS zero. RRC begins CONNECTED because radio attachment precedes the modeled session workflow. The access controls exercise CONNECTED and IDLE; RRC_INACTIVE is not implemented. A logical session can persist while access resources are inactive. A local cell change is a negative control, not a full handover procedure.

B. Transition Relation

The Core begins an operation by allocating a transaction and deadline. Setup targets $(s, a, q) = (1, 1, 1)$; modification requires a committed session and targets $(1, 1, q_C + 1)$; release targets $(0, 0, 0)$. The RAN applies a current request, and the Core commits its target only after processing a current completion. At timeout, the Core clears pending state and retains its previous committed record. The RAN retains any configuration already applied. Table II specifies the principal transitions.

Each session has at most one pending operation. Commit and timeout semantics are experimental choices, not mandatory 3GPP rollback behavior.

TABLE II
Transition Effects in the Implemented EFSM

Event	RAN effect	Core effect
Begin operation	Unchanged	Set pending, transaction, deadline
Apply current request	Apply target	Retain old record
Process completion	Retain target	Commit target; clear pending
Timeout	Retain applied state	Retain old record; clear pending
Stale message	Reject stale request	Reject stale completion
Idle/resume	Toggle access resources	Retain logical session
Recovery	Copy authorization	Retain committed record

C. Coupling Invariants

The verifier evaluates three predicates:

$$I_1 = a_R \wedge \neg s_C, \tag{2}$$

$$I_2 = (a_R \neq a_C) \wedge \neg I_1, \tag{3}$$

$$I_3 = a_R \wedge a_C \wedge (q_R \neq q_C). \tag{4}$$

I_1 denotes an orphan RAN resource; I_2 an access-resource mismatch; and I_3 a QoS-profile mismatch. The orphan predicate takes precedence over a resource mismatch. Both predicates can be absent when access is inactive and the Core retains a logical session. Local cell equality is not an invariant. Absence of these predicates means consistency only with respect to these modeled relationships.

D. Observation-Based Verification

The detector receives complete snapshots containing the key, side, node sequence number, capture timestamp and context record. It never receives injected-fault flags or oracle labels. Each new snapshot replaces the previous view only if its sequence number is greater.

The aware mode applies the following guards. Both sides must be present and no more than 60 ms old. Their stable intervals must overlap:

$$\min(t_R^{cap}, t_C^{cap}) \geq \max(t_R^{chg}, t_C^{chg}). \tag{5}$$

A pending Core operation suppresses evidence before its deadline. If a pending view remains after the deadline, a Core snapshot captured at or after that deadline is required. Accepted contradictory evidence must then persist for a default 12 ms. The monitor emits one alarm per active session/rule candidate and records both contexts and timing evidence.

Stale or incompatible observations reset evidence. They represent unknown state, not a proof of consistency. The common simulator clock is an explicit assumption. With unbounded delay and no additional observations, timeout alone cannot establish permanent message loss. Our detector relies on the modeled committed-state semantics and telemetry guarantees.

E. Recovery Adapter

On an alarm, a custom adapter queries both nodes, refuses a pending operation, and checks versions before writing. It reconciles the RAN session, resource and QoS fields to the committed Core authorization, then queries again to verify convergence. Query and actuation delays each range from 5 to 10 ms, followed by a 3 ms verification delay. A job makes at most two attempts.

Core authority is a modeling choice for these fields. This adapter is not a standardized "Context Sync Request." Successful convergence may remove orphan resources and leave a UE without a usable session. State consistency and service availability are separate outcomes.

IV. Implementation and Experimental Setup

A. Software and Scenario Design

The prototype uses Python and SimPy for virtual-time scheduling, with a Streamlit demonstration interface [7], [8]. Separate modules implement the context model, simulator, verifier, metrics, command-line experiments and plots. The dashboard summarizes detection and recovery metrics, filters per-session event timelines, compares configurations against the precomputed benchmark, and exports run results.

The twelve scenarios are clean operation, benign delay, request-processing failure, QoS/setup/release completion-processing failure, reordered observations, queued high load, observation outage, access idle/resume, local cell change, and unavailable recovery. Request-processing failure can prevent service without creating a contradiction; injection is therefore not itself a positive ground-truth label.

Each run contains three UEs with two sessions each over 540 virtual ms. Default monitor/oracle ticks are 2 ms, polling is 15 ms, and the procedure timeout is 110 ms. Ordinary application-leg delay is uniform over 3–9 ms; benign-delay legs use 18–42 ms. High load adds a shared three-worker processing queue with 18–42 ms service times. Fault eligibility is sampled per session with probability 0.7. This is not an independently sampled per-packet loss rate.

The main suite uses 40 seeds and four configurations: immediate checking, a fixed 25 ms mismatch checker, aware checking without recovery, and aware checking with recovery. This yields $12 \times 40 \times 4 = 1,920$ runs. Independent observation and recovery random-number streams preserve paired node trajectories until intervention. Additional held-out experiments use seeds 101–120, three scenarios, four polling intervals and three grace values, yielding 720 runs.

B. Ground Truth and Metrics

An oracle reads actual node states at each tick and opens a contradiction episode only after pending procedures end. The verifier consumes observations alone. Both use the same semantic predicates, so this evaluates monitoring under the declared model, not independent conformance of those predicates to the full standards.

An alarm is a true positive only if its key and rule match a truth episode and its time lies within that episode. Early alarms during valid transitions are false positives even if a contradiction develops later. Repeat alarms within an already matched episode count separately as duplicates. Precision is $TP/(TP+FP)$ and recall is detected episodes divided by all truth episodes. The false-positive rate divides negative sessions receiving a false alarm by all sessions with no truth episode in that run. Undefined denominators remain N/A; aggregation pools counts.

Detection delay starts at the oracle's episode onset. Fixed probes at 285, 335 and 510 ms fail when the actual context has a stable contradiction. Lead time is the interval from an alarm to the first subsequent failed probe in the same episode. Probes are independent of alarm timing and measure context consistency, not calls or packet delivery. Total mismatch exposure is the sum of episode durations in session-ms. Recovery success counts verified successful jobs divided by all recovery jobs.

TABLE III
Aware Detection Without Recovery, 40 Seeds

Scenario	TP	FP	FN	Delay (ms)	Lead (ms)
Setup completion	167	0	0	21.87	138.15
QoS completion	167	0	0	20.93	18.35
Release completion	167	0	0	22.35	22.67
Queued high load	154	0	0	22.48	98.44
Observation outage	167	0	0	84.36	N/A

V. Results and Discussion

A. Detection Accuracy and Timing

Table III gives aware results with recovery disabled. All listed truth episodes were detected without false alarms in these selected synthetic cases. Perfect scores do not estimate real-network accuracy. Observation outages increase mean delay because the monitor withholds conclusions until adequate observations return.

Lead time is conditional on a future failed probe. The outage scenario has none, giving N/A. Setup probe lead reflects the chosen schedule, not a measured call-drop prediction horizon.

B. Evaluation Questions and Baseline Interpretation

The evaluation addresses three questions: whether procedure-aware evidence avoids benign-transition alarms, how long stable contradictions take to detect, and whether bounded reconciliation reduces contradiction duration. Immediate and fixed-delay checkers use the same semantic rules and observation stream. They are detector ablations, not implementations of the cited KPI-based AI framework. Comparisons therefore isolate timing and observation guards within this simulator.

For the benign-delay control, the immediate, fixed and aware configurations yield 720, 449 and zero false alarms respectively. These are alarm counts. Their false-positive session counts are 240, 225 and zero, demonstrating why alarm totals and session rates must be distinguished. With no truth

TABLE IV
False-Positive Session Counts in Negative Controls

Control	Immediate	Fixed	Aware
Clean	240/240	0/240	0/240
Benign delay	240/240	225/240	0/240
Request failure	240/240	0/240	0/240
Reordered telemetry	240/240	58/240	0/240
Idle and resume	240/240	0/240	0/240
Local cell change	240/240	0/240	0/240

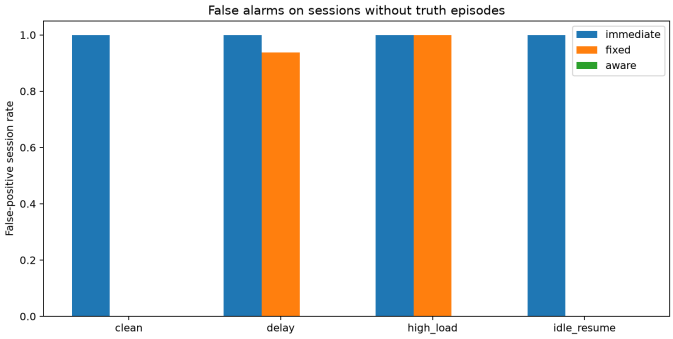


Fig. 1. False-positive session rates for the implemented baselines and aware verifier. Results are synthetic and scenario-specific.

episodes, recall is undefined. With no alarms, aware precision is also undefined, rather than a perfect score. Table IV summarizes additional negative controls.

C. False Positives Under Valid Delay

All 240 sessions in the benign-delay scenario are negative sessions. Immediate checking falsely flags all 240, and the fixed 25 ms baseline flags 225 (93.75%). The aware detector flags none. Its advantage follows from waiting for procedure evidence and compatible observations, rather than treating any sufficiently long mismatch as a failure. Fig. 1 includes additional controls. High-load false-positive rates use only sessions without truth episodes; high load can also cause genuine post-timeout contradictions.

Clean, request-failure, reordered-observation, idle/resume and local-cell controls also produce no aware false alarms.

D. Recovery Effectiveness

Table V compares paired recovery-off/on runs. Each row contains 720 probes. For QoS completion failures, all 167 recovery jobs converge, but 148 probes still fail because some probes precede successful intervention. Therefore 100% job success does not mean that every downstream probe succeeds.

QoS mismatch exposure falls from 15,698 to 6,192 session-ms, a 60.56% reduction. Failed probes fall by 55.69%. These totals aggregate 40 seeds rather than describe one trace. Setup recovery can remove orphan resources and make consistency probes pass without re-establishing a desired session. Observation outages leave little intervention time before later operations and trigger pending/version refusals. An unavailable adapter produces no successful reconciliation.

TABLE V
Recovery Results Across 40 Paired Seeds

Scenario	Failed probes off	Failed probes on	Successful jobs
Setup completion	334	0	167/167
QoS completion	334	148	167/167
Release completion	167	41	167/167
Queued high load	267	21	154/154
Observation outage	334	334	15/167
Adapter unavailable	334	334	0/167

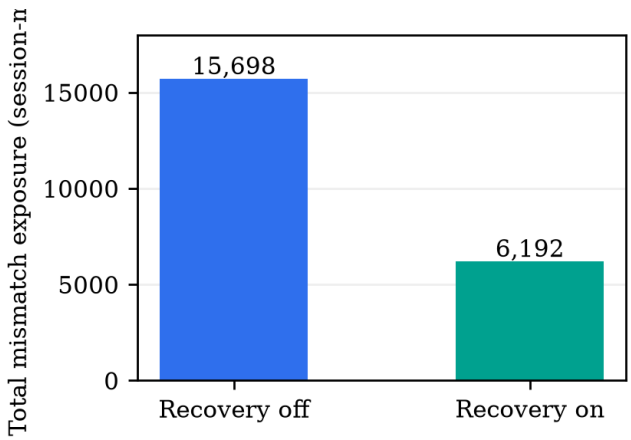


Fig. 2. QoS completion-failure exposure across 40 paired seeds. Recovery reduces the accumulated duration of stable contradictions, including intervals before intervention.

E. Paired Uncertainty Analysis

A new analysis resamples the 40 benchmark seeds as independent clusters, retaining all six sessions and both recovery configurations within each selected seed. We use 10,000 resamples with random seed 20261103 and report percentile 95% confidence intervals. For each resample, reductions are computed from pooled off/on totals, rather than averaging per-run percentages. Detection delay is weighted by the number of matched episodes.

For QoS completion failures, failed-probe reduction is 55.69% (95% interval 53.82–57.63%), and exposure reduction is 60.56% (60.06–61.09%). Mean detection delay is 20.93 ms (20.56–21.28 ms). These intervals describe seed variability under fixed simulator assumptions. They do not account for model error, different networks or untested faults. Session outcomes within a seed are not treated as independent samples. No confidence interval is asserted for universal detector correctness.

F. Timing Sensitivity and Software Checks

The held-out suite varies polling over 5, 15, 30 and 90 ms and grace over 0, 12 and 24 ms for QoS completion failure, benign delay and observation reordering. With 12 ms

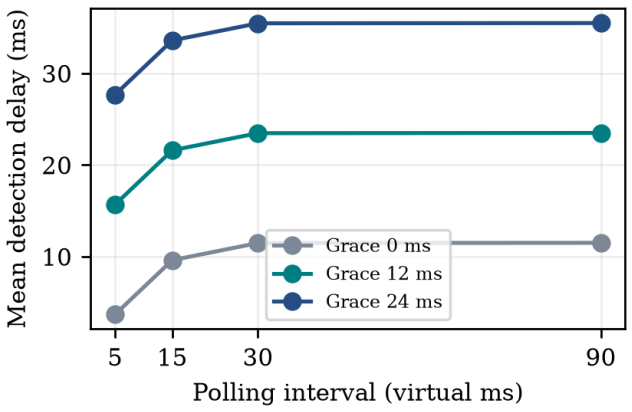


Fig. 3. Held-out QoS completion-failure timing with 20 seeds per point. Event-triggered observations remain enabled at every polling interval.

grace, QoS completion failure yields 77 detected episodes and zero misses at each polling interval. Mean detection delays are 15.69, 21.61, 23.48 and 23.51 ms respectively. Event-triggered snapshots remain enabled, so polling interval is not total observation latency. Fig. 3 shows all twelve timing combinations for the fault scenario. Increasing persistence adds approximately its configured delay. The near-flat 30–90 ms polling result reflects the event-triggered channel and must not be generalized to polling-only telemetry. Benign-delay and reordered-observation controls have zero false alarms across all tested settings.

The implementation passes 49 behavioral and integration tests, including dashboard tests. Recorded core line coverage is 98.9%. The supplied package records passing code and type checks; those checks were not rerun for this report update. The 49 tests were independently rerun on 3 November 2026 using Python 3.12.14 on macOS, with 530 of 536 core statements covered (98.88%). The supplied Windows evidence used Python 3.13.16. All 1,920 benchmark runs reproduced the stored non-runtime metrics to numerical tolerance, and the 720-run sensitivity table matched exactly. This compatibility check does not replace the project's declared Python 3.13 requirement. A review fixed source-hash collection to resolve from the package location, enabling experiment provenance when launched from a different working directory. A Windows launcher invokes the existing environment directly. These software checks support reproducibility, not 3GPP certification.

VI. Limitations and Threats to Validity

The simulator abstracts resource transactions and does not encode full NAS, NGAP, QoS-flow/DRB mapping, independent N4 behavior or handover signaling. RRC access controls are limited. Commit policy, timeouts, delay distributions and Core authority are declared experimental assumptions.

Complete snapshots, reliable identities, monotonic sequences and a common clock simplify observation. Real deployments require instrumented adapters, authenticated telemetry, identity correlation and conservative clock-skew bounds. Packet traces alone may not reveal internal committed state. The shared oracle predicates limit independent validation of semantic correctness.

Six sessions per run and a finite horizon do not establish scalability. Snapshot storage is $O(S)$ for S sessions, but the current checker scans candidate keys inside its session loop and can require $O(S^2)$ work per tick. An indexed candidate structure and measured larger workloads are future work. Local execution time is not production monitoring latency.

Consistency probes do not measure throughput or calls. Recovery can restore agreement by deactivating resources; failed requests can harm availability without desynchronization. The baselines are task ablations, not replicas of the cited AI system.

VII. Conclusion

This project implements and evaluates an explainable verifier for a scoped RAN–Core context model. Fresh and temporally aligned observations, procedure guards and persistence distinguish modeled stable contradictions from valid temporary disagreement. The 2,640-run evaluation demonstrates zero aware false alarms in selected benign controls and lower mismatch exposure when bounded recovery is available. The result supports procedure-aware semantic monitoring within this model, while observation outages and adapter unavailability remain explicit failure boundaries. The next step is independent validation on instrumented RAN/Core software with user-plane demand, clock skew, concurrent operations and genuine mobility procedures.

Reproducibility

The package includes code, tests, a formal model and locked dependencies. Original main results are in `results/final-benchmark/`; the independent rerun is in `results/verified-benchmark/` (seeds 1–40); held-out results are in `results/sensitivity.csv`. The Windows launcher `run.ps1` accepts `app`, `test`, `benchmark` or `sensitivity`. Paired uncertainty results are in `results/evaluation/uncertainty.json`. All reported delays use virtual time. Paths are relative to the project root.

References

[1] Y.-J. Cho et al., "A Framework for AI-Based Large-Scale Time-Series Anomaly Detection in 5G Core Networks," in *Proc. IEEE ICTC*, 2025, doi: 10.1109/ICTC66702.2025.11388910.
[2] 3GPP, "Procedures for the 5G System (5GS); Stage 2," TS 23.502, V16.18.0, secs. 4.3.2–4.3.4.
[3] 3GPP, "NG Application Protocol (NGAP)," TS 38.413, V16.5.0, secs. 8.2.1–8.2.3.
[4] 3GPP, "NG signalling transport," TS 38.412, V16.0.0, 2020, secs. 4.1 and 7.
[5] J. Sid-Otmane, S. Imadali, F. Martelli, and M. Shapiro, "Data Consistency in the 5G Specification," in *Proc. ICIN*, 2020, pp. 110–117, doi: 10.1109/ICIN48450.2020.9059408.
[6] 3GPP, "NR; NR and NG-RAN Overall Description; Stage 2," TS 38.300, V16.2.0, 2020.
[7] SimPy, "Discrete-event simulation framework," [Online]. Available: https://simpy.readthedocs.io/.
[8] Streamlit, "Documentation," [Online]. Available: https://docs.streamlit.io/.